Compte rendu Git

Léo Peintre-Godin[1], Lénaëlle Picard[2], Ismail Hamitouche[3], and Mathis Girard[4]

[1, 2, 3, 4]Université Lumière Lyon 2

March 2026

1 Introduction/contexte

Le groupe est composé de 4 membres : Léo Peintre-Godin, Lénaëlle Picard, Ismail Hamitouche et Mathis Girard. Nous somme tous dans le groupe TD3. Pour ce projet, nous avons décidé de travailler sur le rendu Latex. L'adresse de notre dépôt en ligne est la suivante:`https://github.com/MegaTeamLyon2/MiniProjetGit.git`
Pour la répartition des tâches : Léo a créé le dépôt sur GitHub (A2) et a géré les branches (B1), Lénaëlle a créé la majorité des issues et des Milestones (C1), Ismaïl a intégré des tags (B2) et Mathis a effectué des pull requests (B3).

2 Description des mises en situation

2.1 [A2]

Nous avons décidé de créer notre dépôt en ligne sur GitHub.

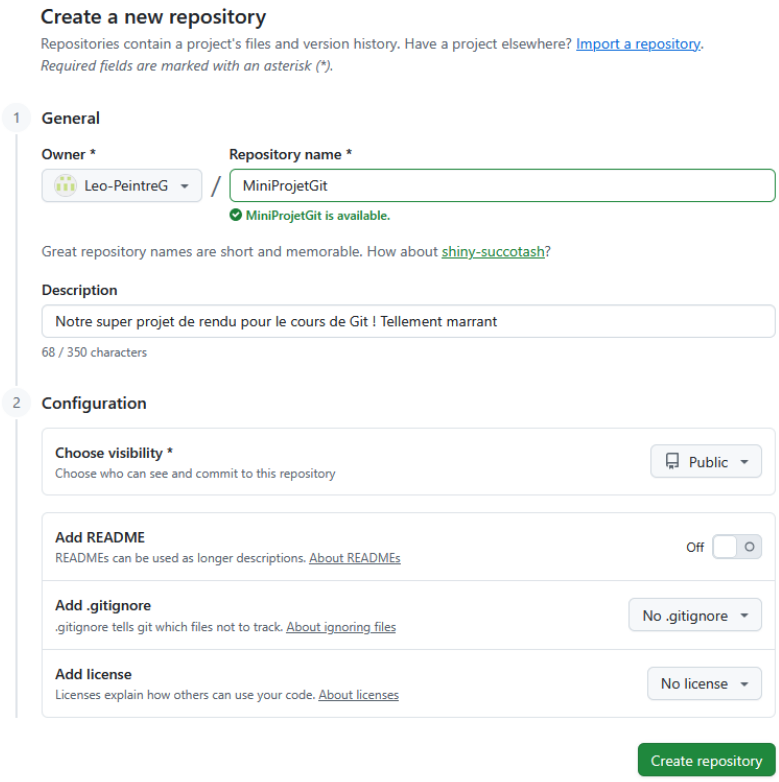

Figure 1: Création du dépôt github

Le dépôt ne contient pas de README, puisqu'il n'y a pas de contenu autre que ce fichier et les différentes images. Le dépôt est par ailleur en public, et le lien se situt dans l'Introduction.

2.2 [B1]

Afin de pouvoir travailler à plusieurs en même temps, nous avons créer un branche dans le projet, nommée "Sans-notes". Comme son nom l'indique, cette brnahce contient le même contenu que cette branche, sans le package "todonotes" permettant de rajouter des petites notes personnalisées.

```
super@PC_SuperSolga MINGW64 ~/OneDrive/Bureau/git2026/MiniProjetGit (main)
$ git branch "Sans-Notes"

super@PC_SuperSolga MINGW64 ~/OneDrive/Bureau/git2026/MiniProjetGit (main)
$ git checkout Sans-Notes
Switched to branch 'Sans-Notes'

super@PC_SuperSolga MINGW64 ~/OneDrive/Bureau/git2026/MiniProjetGit (Sans-Notes)
$ git add .
warning: in the working copy of 'Leo-PeintreG', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'Leo-PeintreG.pub', LF will be replaced by CRLF
the next time Git touches it

super@PC_SuperSolga MINGW64 ~/OneDrive/Bureau/git2026/MiniProjetGit (Sans-Notes)
$ git commit -m "Ajout du pdf compilé de LaTeX, sans annotations."
[Sans-Notes e7e7050] Ajout du pdf compilé de LaTeX, sans annotations.
 4 files changed, 40 insertions(+), 2 deletions(-)
 create mode 100644 Leo-PeintreG
 create mode 100644 Leo-PeintreG.pub

super@PC_SuperSolga MINGW64 ~/OneDrive/Bureau/git2026/MiniProjetGit (Sans-Notes)
$ git push -u origin Sans-Notes
Enumerating objects: 9, done.
```

Figure 2: Tag

2.3 [B2]

Pour créer un tag, nous disposons de plusieurs paramètres. On peut également créer un tag depuis bash directement en lui donnant un nom et un commit précis. La version bash permet de faire un tag sur le commit que l'on veut alors que la version github nous bloque au commit les plus récent. J'ai tout d'abord créé un premier tag avec le tout premier commit effectué sur le projet -¿ c'est la version 0.1.0.

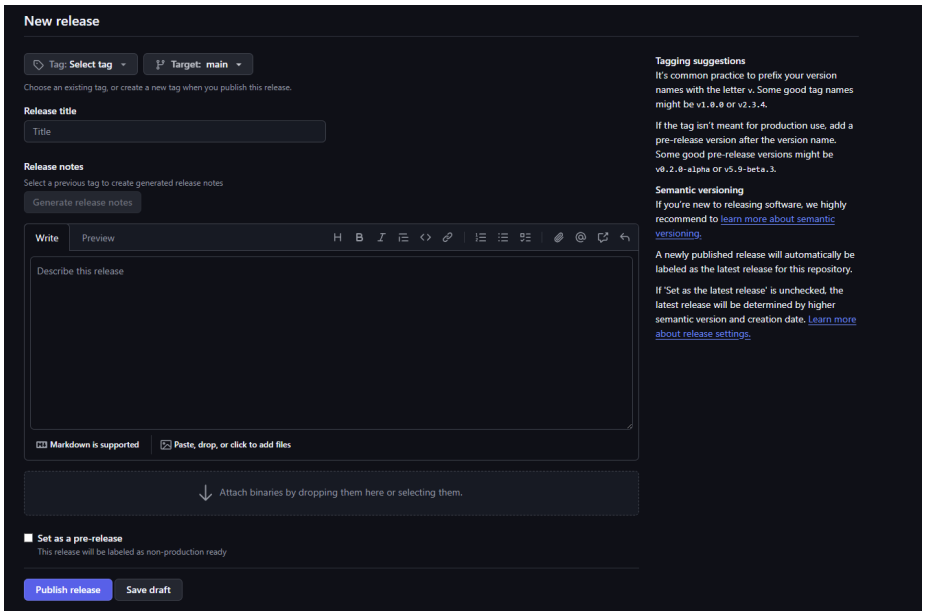

Figure 3: Tag

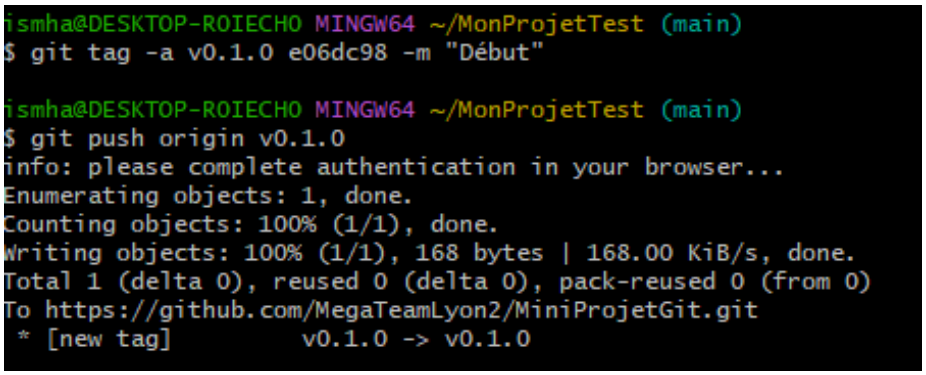

Figure 4: Premier tag

Un release a été associé à ce tag, c'est un pre-release car le projet n'est pas encore utilisable. Pour créer de nouveaux tag et de nouveaux release, il suffit de cliquer sur "Releases" puis sur "Draft a new release".

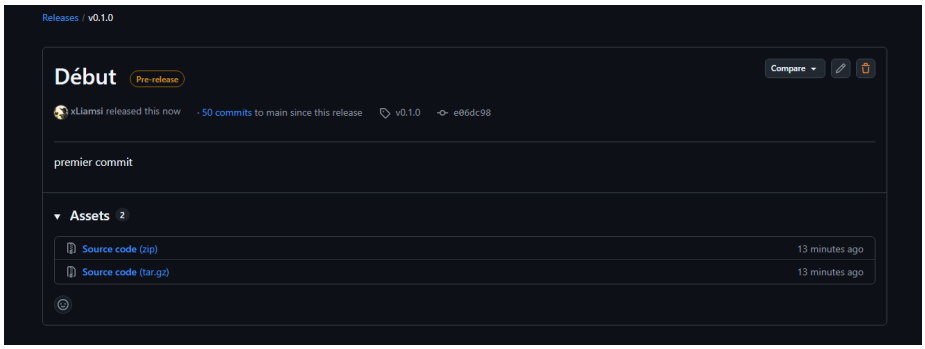

Figure 5: Premier release

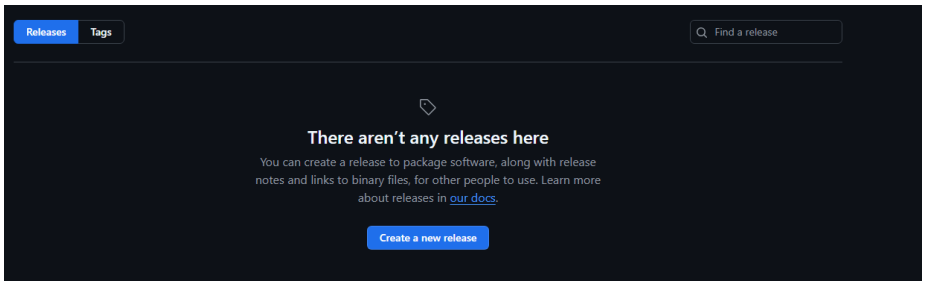

Figure 6: Créer un nouveau release

Par la suite on crée plusieurs tag au fur et à mesure de l'avancement du projet. Par exemple le tag v0.2.0 signifie qu'une nouvelle page a été ajoutée, en l'occurence le Latex. Le tag v0.2.1 signifie qu'il y a eu une modification dans le document Latex mais pas d'ajout de nouvelles pages ou fonctionnalités

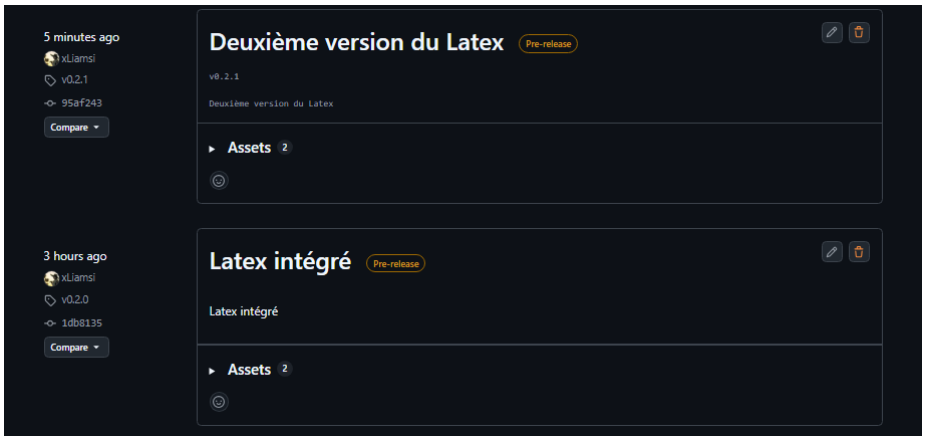

Figure 7: La suite des releases

On fait ca ainsi de suite pour tous les tags en partants de ce principe, lors d'une grosse modification on modifie le dernier chiffre et lors d'une correction mineure on change le dernier chiffre.

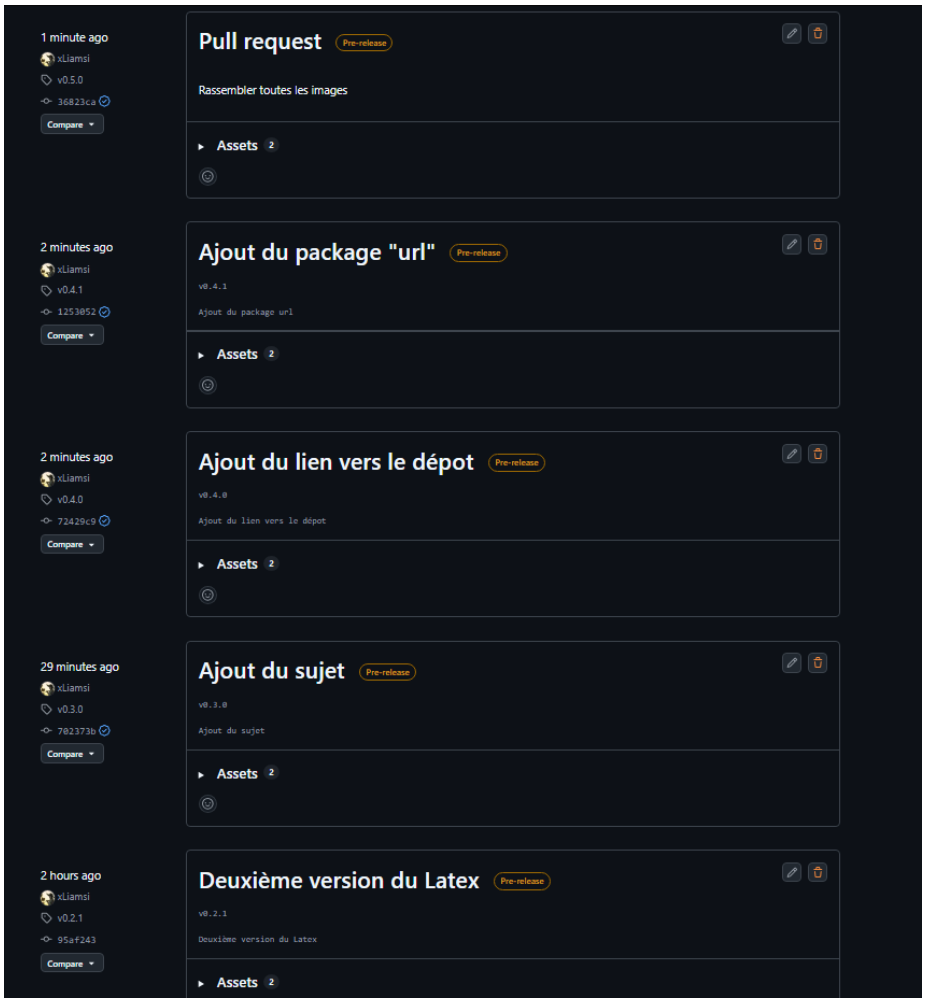

Figure 8: Liste des release

On a aussi la possibilité de push plusieurs tags à la fois depuis le bash sur le Github

```
ismha@DESKTOP-ROIECHO MINGW64 ~/MonProjetTest (main)
$ git tag -a v0.4.0 702373b -m "Ajout du lien vers le dépot"

ismha@DESKTOP-ROIECHO MINGW64 ~/MonProjetTest (main)
$ git tag -a v0.4.1 702373b -m "Ajout du package url"

ismha@DESKTOP-ROIECHO MINGW64 ~/MonProjetTest (main)
$ git push origin --tags
Enumerating objects: 2, done.
Counting objects: 100% (2/2), done.
Delta compression using up to 6 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 236 bytes | 236.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
```

Figure 9: Push plusieurs tag à la fois

On peut égalemet supprimer des tags directement dans le bash

```
ismha@DESKTOP-ROIECHO MINGW64 ~/MonProjetTest (main)
$ git tag -d v0.4.1
Deleted tag 'v0.4.1' (was 4b28ad7)

ismha@DESKTOP-ROIECHO MINGW64 ~/MonProjetTest (main)
$ git push origin --delete v0.4.1
To https://github.com/MegaTeamLyon2/MiniProjetGit.git
 - [deleted]         v0.4.1
```

Figure 10: Comment supprimer un tag dans bash

Le processus de tag peut se dérouler en beaucoup plus d'itérations qu'ici présent, il est préférable de ne pas en mettre trop afin de ne pas saturer les release et les tags.
On peut désormais associé le tag v1.0.0 au dernier commit effectuer afin de montrer que la version peut être présentée.

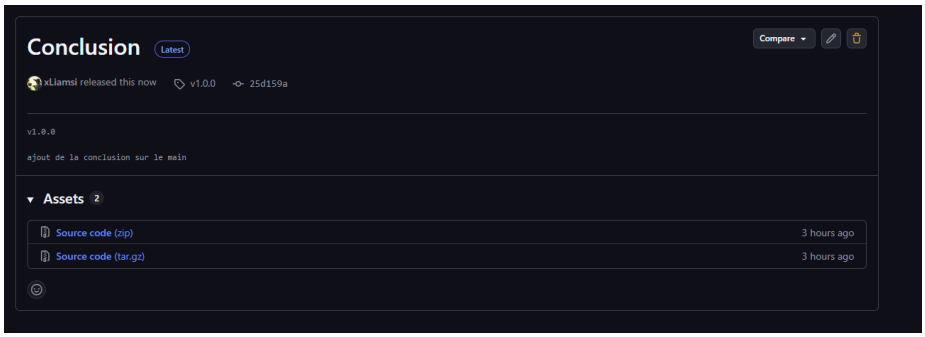

Figure 11: Tag de la v1.0.0

2.4 [B3]

Pour créer un pull request on passe directement par l'interface Github [12] Qui va intégrer ce que j'ai fait plus tôt sur Bash [13]

Figure 12: image 1

```
(feature/Mathis)
$ git config --global user.email arcaneice9@gmail.com

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git config --global user.name Mathis

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git commit -m "Ajout section pour les pull request dans le rapport"
On branch feature/Mathis
nothing to commit, working tree clean

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git push -u origin feature/Mathis
info: please complete authentication in your browser...
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/Mathis' on GitHub by visiting:
remote:      https://github.com/MegaTeamLyon2/MiniProjetGit/pull/new/feature/Mat
his
remote:
To https://github.com/MegaTeamLyon2/MiniProjetGit.git
 * [new branch]      feature/Mathis -> feature/Mathis
branch 'feature/Mathis' set up to track 'origin/feature/Mathis'.

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ nano oui.txt

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git add oui.txt
warning: in the working copy of 'oui.txt', LF will be replaced by CRLF the next
time Git touches it

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git push oui.txt
fatal: invalid gitfile format: oui.txt
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git commit -m oui.txt
[feature/Mathis 42078e9] oui.txt
 1 file changed, 2 insertions(+)
 create mode 100644 oui.txt

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git push oui.txt
fatal: invalid gitfile format: oui.txt
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git add .

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git commit -m "commentaire"
On branch feature/Mathis
Your branch is ahead of 'origin/feature/Mathis' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 286 bytes | 286.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/MegaTeamLyon2/MiniProjetGit.git
   05b84ee..42078e9  feature/Mathis -> feature/Mathis

arcan@DESKTOP-BOJQEUS MINGW64 ~/Documents/J'AIME PAS LA FAC/GitHub/MiniProjetGit
(feature/Mathis)
$ |
```

Figure 13: image 2

2.5 [C1]

En intégrant des issues et des milestones il est plus simple d'organiser le projet.
On peut confier les issues à certaines personnes du projet.
Les issues et milestones se font depuis le site github. Lorsque les issues sont effectuées elles peuvent être fermées.
Les milestones permettent de mieux organiser les issues (par exemple par thèmes).

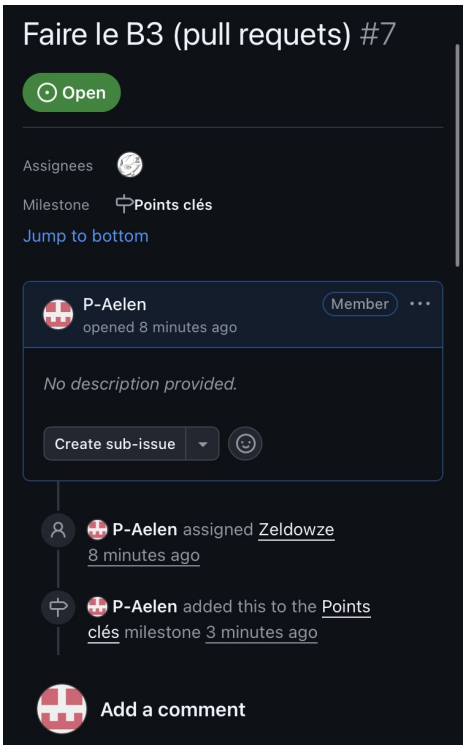

Figure 14: Création d'une issue

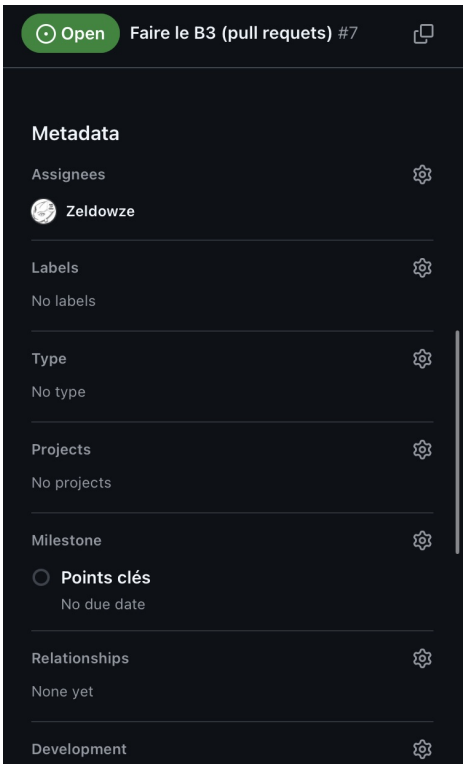

Figure 15: Paramètres ajoutés aux issues

Ici [14, 15], Lénaëlle a créé une issue et l'a assignée à Mathis pour qu'il fasse le B3. L'issue a également été assignée au milestone "Points clés" pour qu'on puisse retrouver plus facilement quels points clés il fallait encore faire.

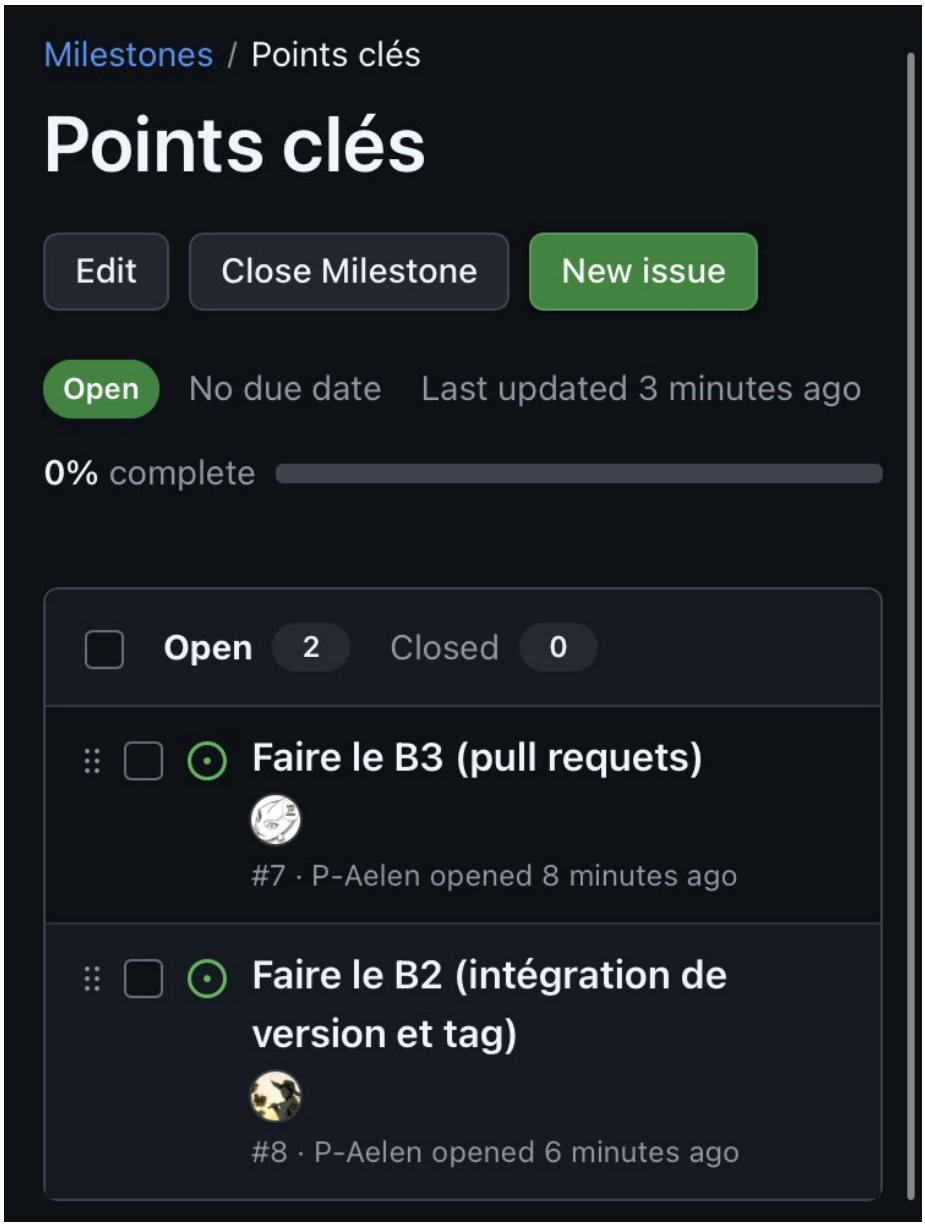

Figure 16: Organiser les issues avec les milestones

3 Conclusion

En conclusion, ce mini-projet nous a permis de mettre en pratique les notions abordées durant les séances de TD, notamment la gestion collaborative avec

Git et GitHub. Nous avons pu constituer un groupe efficace et coordonner nos contributions grâce aux branches, aux *pull requests* et à l'utilisation d'*issues* pour le suivi des tâches. Le dépôt Git et les tags de version associé ont illustré de manière concrète la mise en place d'un projet structuré et collaboratif.

Malgré quelques difficultés techniques, notamment un problème de connexion rencontré par un membre du groupe qui a dû cloner le projet à plusieurs reprises, l'ensemble des objectifs a été atteint. Cette expérience nous a permis de mieux comprendre les bonnes pratiques de travail en équipe et de gestion de versions, tout en renforçant notre autonomie face aux outils professionnels de développement.